

hermetic_ftp_run.sh — companion guide (§11.4.18)

Revision: 2 **Last modified:** 2026-07-02T00:00:00Z **Status:** H2.ftp for the hermetic WireGuard test harness ([design](#)) — the second §11.4.52 **promotion** of an operator-gated protocol test to autonomous (after the [Chromecast eureka leg](#)). Authority: inherits constitution/Constitution.md per §11.4.35.

Overview

tests/vpn_lan/hermetic_ftp_run.sh promotes the **FTP leg** of the operator-gated tests/vpn_lan/ftp_sftp_webdav.sh to run **autonomously** (§11.4.52). Inside one unshare -Urnm it: (1) stands up the H0-full kernel-WireGuard tunnel between two unprivileged netns; (2) runs a **real, pure-python-stdlib FTP server** in netns B bound to the WG-only overlay 10.10.0.2:2121; (3) points the bridge contract (tests/lib/sword_bridge.sh) at it; (4) runs the **UNMODIFIED** protocol test *inside* netns A, where its own bridge_require gate flips SKIP→UP and the FTP leg does a **real passive directory listing** over the encrypted tunnel and produces a genuine PASS with captured evidence — no operator, no podman, no live VPN server. The harness **additionally** content-verifies a **byte fetch (RETR)** of a fresh nonce file over the tunnel itself (§11.4.107(9)) — the promoted test's own fetch is best-effort + ungated (its PASS is listing-only), so the byte-transfer proof is earned by the harness self-check, not claimed on the test's behalf.

Zero host installs (§11.4.122): python has no stdlib FTP *server* (only the ftplib client), so the harness embeds a ~85-line pure-stdlib FTP server answering exactly the command set curl --ftp-pasv drives: USER/PASS(anonymous)/SYST/TYPE/PWD/CWD/SIZE/FEAT/OPTS/EPSV/PASV/LIST/RETR/QUIT.

Why it works over a point-to-point WireGuard link (§11.4.111)

FTP passive mode is the tricky part. A naïve server advertises 127,0,0,1 in its 227 Entering Passive Mode reply; the client (curl, in netns A) then dials its own loopback and the data channel dies. This server advertises the **overlay address 10.10.0.2** (PASV) — or

uses **EPSV** 229 (|||port|), which carries no IP so the client reuses the control connection's peer (already 10.10.0.2 over the tunnel). Both are handled; curl prefers EPSV. The data connection then traverses wg0, inside the WG allowed-ips 10.10.0.1/32 ↔ 10.10.0.2/32.

How the bridge contract is pointed at the hermetic peer

var	hermetic value
HELIX_SWORD_DIR	the served dir (non-empty)
HELIX_BRIDGE_CONNECT / _DISCONNECT	true
HELIX_BRIDGE_HEALTH	a real TCP probe of 10.10.0.2:2121 over the tunnel
HELIX_BRIDGE_SUBNET	10.10.0.0/24
HELIX_BRIDGE_HOST	10.10.0.2
HELIX_VPN_FTP_URL	ftp://10.10.0.2:2121/ (anonymous — no credentials, §11.4.10)

HELIX_BRIDGE_MODE=hermetic documents intent; the library realises the promotion through the contract vars above. The SFTP + WebDAV legs of the same test SKIP honestly (their vars unset) — exactly as designed.

Usage

```
tests/vpn_lan/hermetic_ftp_run.sh # PASS / SKIP /  
FAIL  
FT_MUT=empty tests/vpn_lan/hermetic_ftp_run.sh # §1.1 golden-bad
```

Evidence: qa-results/vpn_lan/hermetic_ftp/<UTC-ts>_<pass|mut>_<pid>/run.evidence (the promoted test also writes its own qa-results/vpn_lan/phase3/.../ftp/).

Anti-bluff design

The normal PASS is emitted **only** when the promoted ftp_sftp_webdav.sh exits 0 **and** its stdout carries a real ^PASS:.*FTP passive line — a SKIP-only run can never satisfy that. The **golden-bad** (FT_MUT=empty) makes the FTP server serve an **empty directory listing**; the promoted test then cannot list, so it **SKIPS** (^SKIP:.*FTP passive, never PASS). The harness reports PASS on

the mutation **only** when it confirms that suppression — proving a real PASS provably requires a real non-empty listing+fetch over the tunnel, not a rubber-stamp (§11.4.107(10) / §11.4.68).

Self-fetch freshness cross-check (§11.4.107 not-stale).

Before running the promoted test, the harness lists ftp://10.10.0.2:2121/ **itself** over the tunnel and asserts THIS run's fresh nonce filename is present (normal) / the listing is empty (golden-bad). This ties the eventual PASS to a real passive round-trip of *our* data over the encrypted overlay, decoupled from the downstream grep string and forbidding a stale/cached peer. A **coupling-contract guard** (grep -q 'FTP passive' "\$TEST") makes a future rename of the promoted leg fail with a clear diagnostic here.

Content-verified byte fetch (§11.4.107(9) full-reference oracle). On the normal path the harness then RETRs the nonce file ftp://10.10.0.2:2121/<nonce>.txt by its exact (CRLF-free, self-constructed) name over the tunnel and asserts the returned bytes equal the known payload ftp-payload-<nonce>. This is what genuinely EARNs the "+fetch" claim — a real byte transfer over the encrypted tunnel, content-checked against ground truth — rather than relying on the promoted test's own fetch, which is best-effort and NOT part of its PASS gate (its listing is CRLF-terminated, so the test's awk-extracted filename carries a trailing \r and curl rejects the URL — harmless, because that fetch is ungated; the harness proves the fetch on its own).

Host-safety self-bounding. The FTP server runs under timeout -k 2 90 *inside* the netns so an outer-SIGKILL orphan self-terminates (§12); the outer timeout 70 reaps the whole unshare tree first, the inner self-bound is belt-and-suspenders.

Ambient-env determinism (§11.4.50). Before invoking the promoted test the harness unsets the SFTP + WebDAV leg contract vars (and the FTP _USER/_PASS/_ACTIVE_CMD auxiliaries), so a leftover ambient-shell export cannot make a sibling leg attempt a (from-netns unreachable) real connection and flip the promoted test's exit code into a spurious FAIL. This is a false-negative guard only — it can never manufacture a bluff PASS — and pins the promoted-test environment to exactly the one configured leg.

Captured evidence (verified 2026-07-02)

Normal run embedded the promoted test's own output:

```
ftp_sftp_webdav: sword bridge UP – running live FTP/SFTP/WebDAV
checks (subnet=10.10.0.0/24 host=10.10.0.2)
PASS: FTP passive directory-list + fetch (VPN server)
```

[evidence: .../phase3/.../ftp/roundtrip.evidence]
FT_PASS: the UNMODIFIED ftp_sftp_webdav.sh FTP leg produced a REAL PASS over the hermetic WireGuard tunnel

3/3 deterministic (§11.4.50). Golden-bad → the real test SKIPped on the empty listing (network_unreachable_external), the teeth fired.

Honest scope (§11.4.6 / §11.4.3)

Proves the **FTP passive listing+fetch logic** autonomously over an encrypted tunnel against a controlled peer. It does NOT prove a real FTP server on the real Mullvad topology (that stays the §11.4.3 operator-gated confirmation), and it does not promote the SFTP leg (needs sshd / an sftp client — both absent, so operator-gated) or the WebDAV leg (routes through the data-plane Squid — see hermetic_webdav_run.md).

Prerequisites / SKIP

bash, unshare+nsenter, iproute2 ip (wireguard link type), wg, host wireguard kernel module, python3 (stdlib only), curl, timeout, plus tests/vpn_lan/ftp_sftp_webdav.sh. Any missing → honest SKIP: (§11.4.3). Process-headroom guard SKIPS on a starved host (§12).

Security (§11.4.10)

WireGuard private keys: mode-0600 mktemp inside the namespace, used by path, removed on exit, never logged. FTP is **anonymous** — no credentials anywhere. The served file is a per-run nonce (fresh value, §11.4.107 not-stale).

Underlay-sniff non-leak differential (§11.4.107 / FINDINGS §7.1)

During the round-trip the harness captures on the underlay veth0 (rootless AF_PACKET; tcpdump fallback; honest §11.4.3 SKIP if neither) and asserts BOTH (a) WG ciphertext present — a type-4 0x04 datagram to the WG listen port :51820 — AND (b) the per-run FTP marker (\$NONCE, the listing filename / RETR body) is ABSENT in the raw underlay bytes. Verbatim single-source clone of the substrate analyzer (_emit_an_py, ethertype-guarded). The load-bearing golden-bad **SNIFF_MUT=plain** emits \$NONCE as cleartext UDP to the discard port 10.9.0.2:9 (distinct from the §11.4.111 negative-control port :2121, so NEG-OK stays valid) → ONLY the

plaintext-absent assertion flips to FAIL while ciphertext stays present, proving the sniff is not a tautology (§11.4.107(10)).
Landed 85d8b32, independent review a1dca6fd GO.

Related

- [hermetic_bridge_run.md](#) — the H2 template (Chromecast eureka leg).
- [hermetic_wg_roundtrip.md](#) — the H0-full tunnel this reuses.
- [hermetic_netns_poc.md](#) — the veth substrate.
- `tests/vpn_lan/ftp_sftp_webdav.sh` — the promoted protocol test.

Last verified

2026-07-02 (host: ALT Linux kernel 6.12, wireguard module, wireguard-tools 1.0.20210914, python3 stdlib, curl).